European Soccer Database

Final Project · Deliverable 4 · soccer_proj

Emmanuel Arthur · Database Class · Spring 2026

Dataset overview

Source: Kaggle European Soccer Database (SQLite export → CSV → PostgreSQL)

Scope: 11 countries, 11 leagues, ~26K matches (2008–2016), teams, players, attributes

Business context: same kind of data clubs, sports media, and analytics vendors use for reporting

Schema soccer_proj with 7 base tables + views for common joins

Total loaded rows across all tables: 222,796

Why I chose this dataset

I have followed European club soccer since childhood (Man United, then Bayern Munich)

The domain is familiar: leagues, seasons, home/away, points, streaks, betting odds

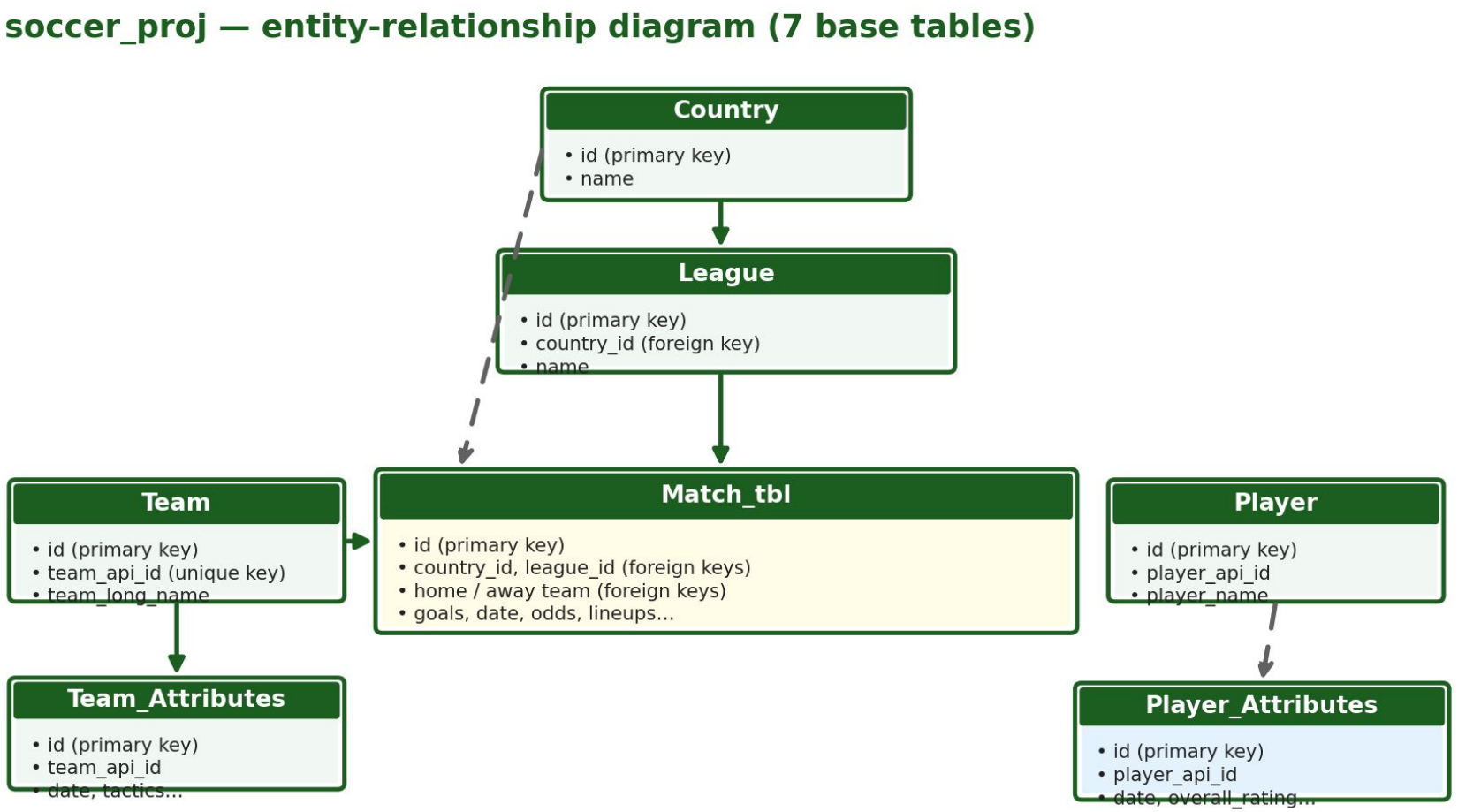
Business-style questions: league scoring trends, defensive strength, player development, odds vs results

Good mix of dimensions (country, league, team, player) and facts (matches, ratings)

Graduate requirement: 30 English questions that map to real SELECT workloads

ER diagram — soccer_proj

Initial and final use the same 7 tables



Data model for business reporting: Country/League/Team/Player = who & where; Match_tbl = transactions (each game); attributes = snapshots over time. Views = ready-made reports.

Hardest part

Loading and typing: SQLite → CSV → Postgres with consistent column names and NULL handling

Match_tbl is very wide (lineup API IDs, many bookmaker columns) — easy to get joins wrong

Writing correct multi-step SQL (common table expressions, window functions) for streaks and lineup-based ratings

Deciding which foreign keys to enforce vs document-only when source data has orphan keys

Keeping 30 project questions aligned with what the data actually supports

What I learned

Design first: ERD and keys matter before writing dozens of queries

JOIN choice changes results (INNER vs LEFT) — always check row counts

Views encapsulate repeated 4–5 table joins; indexes help filter/join columns on Match_tbl

Window functions (partition rows per team/season, compare to prior rows) for streaks and trends

Documentation + reproducible load scripts save time at deliverable deadline

SQL confidence now

Comfortable with the most common SQL: SELECT, JOINs, GROUP BY, HAVING, subqueries, CASE

Comfortable creating views (for example league_match_summary_view, match_context_view)

Overall: much more confident than at the start of the semester — I can debug by counting rows

DBeaver plus the EXPLAIN command helped me see when a query execution plan looked expensive

What I would do next (more time)

Enforce more foreign keys after data cleaning; add CHECK constraints on goals and dates

Materialized views or summary tables for league/season dashboards

More indexes tuned using EXPLAIN on the slowest of the 30 queries

Data quality report: missing lineups, duplicate player identifiers, odds outliers

Optional: BI dashboard (Tableau/Power BI) on views for non-technical stakeholders

DEMO: Tables + row counts

DBeaver · schema soccer_proj

Demo these core tables (foreign key relationships) in DBeaver — columns + row count:

Country — 11 rows (parent; League.country_id → Country.id)

League — 11 rows (country_id foreign key; Match_tbl.league_id → League.id)

Team — 299 rows (home/away team_api_id on Match_tbl)

Match_tbl — 25,979 rows (hub: country_id, league_id, home/away teams, goals)

All 7 tables combined — 222,796 rows total

DEMO: Intermediate query (Q16)

Joins + GROUP BY + AVG (no CASE)

Question: Which countries had the highest average goals per match?

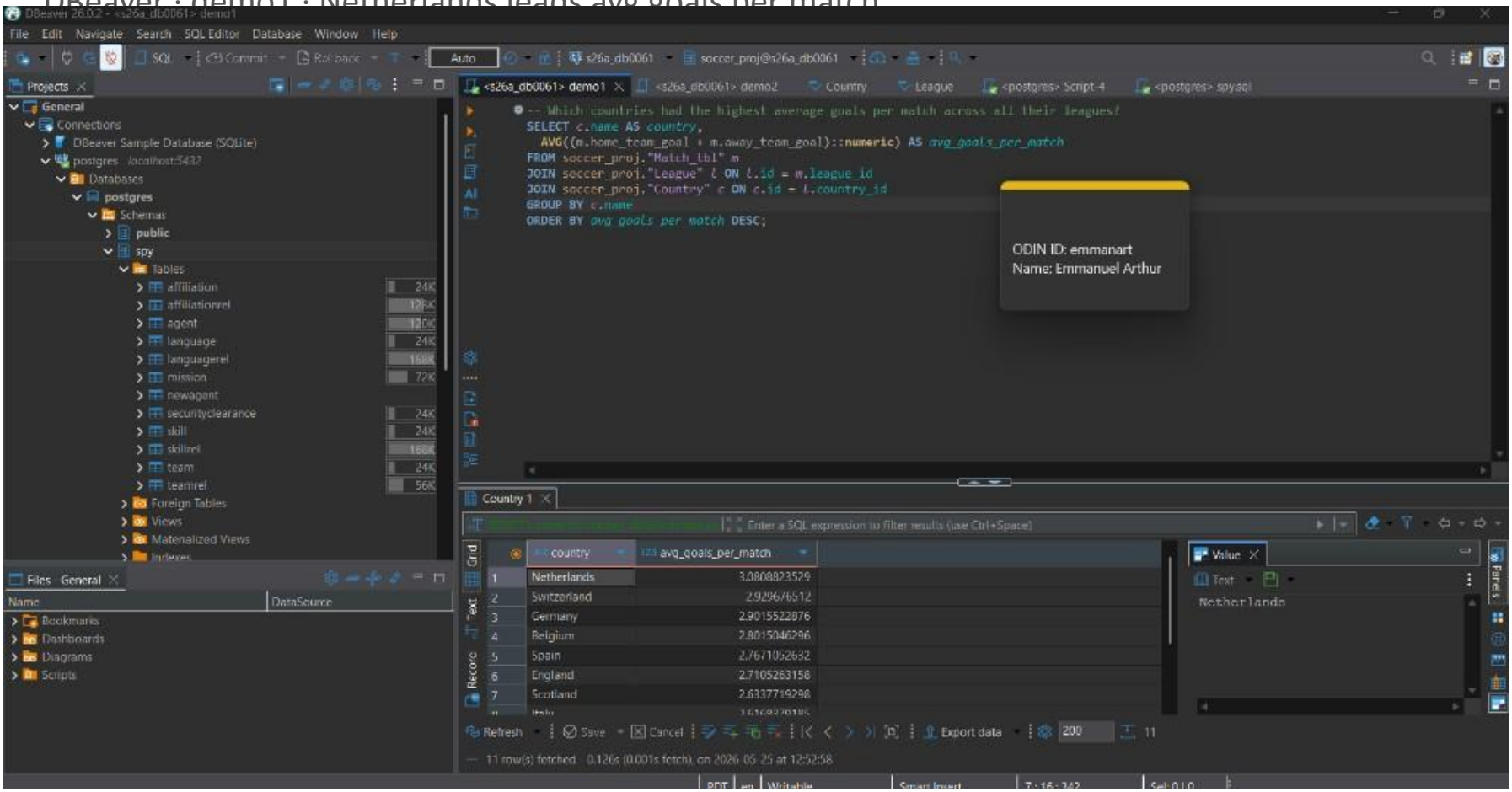
Tables: Match_tbl → League → Country · INNER JOIN, GROUP BY, AVG (no CASE)

SQL (copy into DBeaver on soccer_proj):

```
SELECT c.name AS country,
  AVG((m.home_team_goal + m.away_team_goal)::numeric) AS avg_goals_per_match
FROM soccer_proj."Match_tbl" m
JOIN soccer_proj."League" l ON l.id = m.league_id
JOIN soccer_proj."Country" c ON c.id = l.country_id
GROUP BY c.name
ORDER BY avg_goals_per_match DESC;
```

DEMO Q16 — Query results

DBeaver · demo1 · Netherlands leads avg goals per match



DEMO: Advanced query (Q18)

Join with OR + conditional aggregates

Question: Which teams most frequently kept clean sheets?

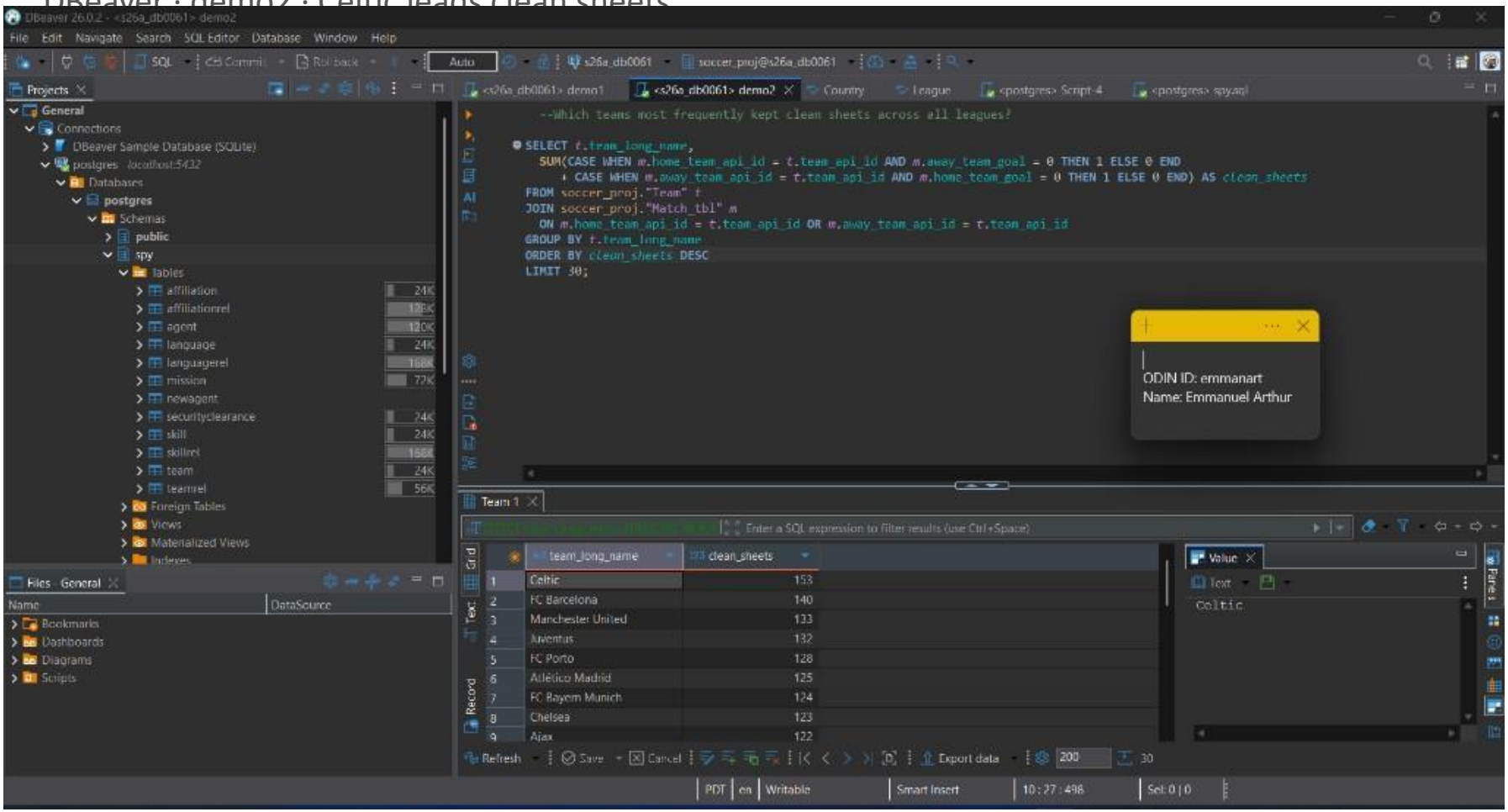
Tables: Team + Match_tbl · JOIN with OR · two CASE inside SUM · GROUP BY

SQL (copy into DBeaver on soccer_proj):

```
SELECT t.team_long_name,
  SUM(CASE WHEN m.home_team_api_id = t.team_api_id AND m.away_team_goal = 0 THEN 1 ELSE 0 END
     + CASE WHEN m.away_team_api_id = t.team_api_id AND m.home_team_goal = 0 THEN 1 ELSE 0 END) AS clean_sheets
FROM soccer_proj."Team" t
JOIN soccer_proj."Match_tbl" m
  ON m.home_team_api_id = t.team_api_id OR m.away_team_api_id = t.team_api_id
GROUP BY t.team_long_name
ORDER BY clean_sheets DESC
LIMIT 30;
```

DEMO Q18 — Query results

DBeaver · demo2 · Celtic leads clean sheets



Business applications

Why this project matters outside class

Club / league operations: season KPIs (goals, clean sheets, home vs away) for coaching and scouting

Media & broadcasting: compare markets (e.g. avg goals by country) for scheduling and storytelling

Betting & risk (data in Match_tbl): historical odds vs outcomes; favorite win rates

Talent / HR analog: Player_Attributes over time ≈ employee skill ratings for hiring and development

Same database patterns as retail (product sales by region) or finance (metrics over time)

Thank you

Questions?

Deliverable 4 write-up + video link in submission